Document Control

Document ID	TAGIC-IR-GL -001
Title	Application API and Secure Coding Guidelines
Version	1.5
Author(s)	Cynthia Saldanha
Release Date	21st-Jan-2021

Revision History

Sr. No.	Revision No.	Effective Date	Change Description	Reviewed By
1	1.0	January 21, 2021	Creation	Sanjay Pai
2	1.1	February 2, 2022	Annual Review no change	Sanjay Pai
3	1.2	February 14, 2023	Annual review, format change and inclusion of sections 8, 9, 10, 11,12	Dhiraj Ranka
4	1.3	February 16, 2024	Annual review with no change	Dhiraj Ranka
5	1.4	February 05, 2025	Annual review with no change	Dhiraj Ranka
6	1.5	January 30,2026	Annual review with no change	Satyanandan Atyam

Approvals History

Sr. No.	Decription	Name	Signature	Date
1	Updated By	Gajendra Sinha- Chief Manager	Approval on Mail	28th-Jan-2026
2	Reviewed By	Satyanandan Atyam - CRO	Approval on Mail	28th-Jan-2026
3	Approved By	Ashish Mittal - CTO	Approval on Mail	28th-Jan-2026

Document Definition

This document serves the purpose of stating roles and responsibility for Application API and Secure Coding Guidelines of the organization.

Note: The controlled master copy of this document is on the shared drive of organization. Printed copies are not controlled. If you are working from a printed copy, please verify with the CISO on version to ensure it is the latest revision.

Table of Contents

1. Purpose of the Document

This guideline document outlines the secure coding standards for the applications and API that are developed within TATA AIG environment as per environment/ platform/ business/ function's applicability to the project. It also defines the coding requirements to meet security standards for optimal control. Checklist of the most important security countermeasures when designing, testing, and releasing an application.

2. Hardening Points Implementation Process:

Hardening Points implemented only after discussion and confirmation from respective application owners i.e., to avoid any adverse impact on application functionality.

3. Hardening Parameters:

Below parameters should be followed during development within TATA AIG environment but not just limited to this, OWASP secure coding guideline should also be followed.

3.1 **Input Validation:**

- Conduct all data validation on a trusted system (e.g., The server)
- Identify all data sources and classify them into trusted and untrusted. Validate all data from untrusted sources (e.g., Databases, file streams, etc.)
- There should be a centralized input validation routine for the application
- Specify proper character sets, such as UTF-8, for all sources of input
- Encode data to a common character set before validating (*Canonicalize*)
- All validation failures should result in input rejection
- Determine if the system supports UTF-8 extended character sets and if so, validate after UTF-8 decoding is completed
- Validate all client provided data before processing, including all parameters, URLs and HTTP header content (e.g., Cookie names and values). Be sure to include automated post backs from JavaScript, Flash or other embedded code
- Verify that header values in both requests and responses contain only ASCII characters
- Validate data from redirects (An attacker may submit malicious content directly to the target of the redirect, thus circumventing application logic and any validation performed before the redirect)
- Validate for expected data types
- Validate data range

- Validate data length
- Validate all input against a "white" list of allowed characters, whenever possible
- If any potentially *hazardous characters* must be allowed as input, be sure that you implement additional controls like output encoding, secure task specific APIs and accounting for the utilization of that data throughout the application. Examples of common hazardous characters include: < > " ' % () & + \ \' \"
- If your standard validation routine cannot address the following inputs, then they should be checked discretely
  o Check for null bytes (%00)
  o Check for new line characters (%0d, %0a, \r, \n)
  o Check for “dot-dot-slash" (./ or ..\) path alterations characters. In cases where UTF-8 extended character set encoding is supported, address alternate representation like: %c0%ae%c0%ae/

(Utilize *canonicalization* to address double encoding or other forms of obfuscation attacks)

3.2 Output Encoding:

- Conduct all encoding on a trusted system (e.g., The server)
- Utilize a standard, tested routine for each type of outbound encoding
- *Contextually output encode* all data returned to the client that originated outside the application's *trust boundary*. *HTML entity encoding* is one example, but does not work in all cases
- Encode all characters unless they are known to be safe for the intended interpreter
- Contextually *sanitize* all output of un-trusted data to queries for SQL, XML, and LDAP
- *Sanitize* all output of un-trusted data to operating system commands

3.3 Authentication and Password Management:

- Require authentications for all pages and resources, except those specifically intended to be public
- All authentication controls must be enforced on a trusted system (e.g., The server)
- Establish and utilize standard, tested, authentication services whenever possible
- Use a centralized implementation for all authentication controls, including libraries that call external authentication services
- Segregate authentication logic from the resource being requested and use redirection to and from the centralized authentication control
- All authentication controls should fail securely
- All administrative and account management functions must be at least as secure as the primary authentication mechanism
- If your application manages a credential store, it should ensure that only cryptographically strong one-way salted hashes of passwords are stored and that the

table/file that stores the passwords and keys is write-able only by the application. (Do not use the MD5 algorithm if it can be avoided)

- Data should also be encrypted in transit using strong cryptographically strong encryption algorithms
- Password hashing must be implemented on a trusted system (e.g., The server).
- Validate the authentication data only on completion of all data input, especially for *sequential authentication* implementations
- Authentication failure responses should not indicate which part of the authentication data was incorrect. For example, instead of "Invalid username" or "Invalid password", just use "Invalid username and/or password" for both. Error responses must be truly identical in both display and source code
- Utilize authentication for connections to external systems that involve sensitive information or functions
- Authentication credentials for accessing services external to the application should be encrypted and stored in a protected location on a trusted system (e.g., The server). The source code is NOT a secure location
- Use only HTTP POST requests to transmit authentication credentials
- Only send non-temporary passwords over an encrypted connection or as encrypted data, such as in an encrypted email. Temporary passwords associated with email resets may be an exception
- Enforce password complexity requirements established by policy or regulation. Authentication credentials should be sufficient to withstand attacks that are typical of the threats in the deployed environment. (e.g., requiring the use of alphabetic as well as numeric and/or special characters)
- Enforce password length requirements established by policy or regulation. Eight characters is commonly used, but 16 is better or consider the use of multi-word pass phrases
- Password entry should be obscured on the user's screen. (e.g., on web forms use the input type "password")
- Enforce account disabling after an established number of invalid login attempts (e.g., five attempts are common). The account must be disabled for a period of time sufficient to discourage brute force guessing of credentials, but not so long as to allow for a denial-of-service attack to be performed
- Password reset and changing operations require the same level of controls as account creation and authentication.
- Password reset questions should support sufficiently random answers. (e.g., "favourite book" is a bad question because “The Bible” is a very common answer)
- If using email-based resets, only send email to a pre-registered address with a temporary link/password
- Temporary passwords and links should have a short expiration time
- Enforce the changing of temporary passwords on the next use

- Notify users when a password reset occurs
- Prevent password re-use
- Passwords should be at least one day old before they can be changed, to prevent attacks on password re-use
- Enforce password changes based on requirements established in policy or regulation. Critical systems may require more frequent changes. The time between resets must be administratively controlled
- Disable "remember me" functionality for password fields
- The last use (successful or unsuccessful) of a user account should be reported to the user at their next successful login
- Implement monitoring to identify attacks against multiple user accounts, utilizing the same password. This attack pattern is used to bypass standard lockouts, when user IDs can be harvested or guessed
- Change all vendor-supplied default passwords and user IDs or disable the associated accounts
- Re-authenticate users prior to performing critical operations
- Use *Multi-Factor Authentication* for highly sensitive or high value transactional accounts
- If using third party code for authentication, inspect the code carefully to ensure it is not affected by any malicious code

3.4 **Session Management:**

- Use the server or framework's session management controls. The application should only recognize these session identifiers as valid
- Session identifier creation must always be done on a trusted system (e.g., The server)
- Session management controls should use well vetted algorithms that ensure sufficiently random session identifiers
- Set the domain and path for cookies containing authenticated session identifiers to an appropriately restricted value for the site
- Logout functionality should fully terminate the associated session or connection
- Logout functionality should be available from all pages protected by authorization
- Establish a session inactivity timeout that is as short as possible, based on balancing risk and business functional requirements. In most cases it should be no more than several hours
- Disallow persistent logins and enforce periodic session terminations, even when the session is active. Especially for applications supporting rich network connections or connecting to critical systems. Termination times should support business

requirements, and the user should receive sufficient notification to mitigate negative impacts

- If a session was established before login, close that session and establish a new session after a successful login
- Generate a new session identifier on any re-authentication
- Do not allow concurrent logins with the same user ID
- Do not expose session identifiers in URLs, error messages or logs. Session identifiers should only be in the HTTP cookie header. For example, do not pass session identifiers as GET parameters
- Protect server-side session data from unauthorized access, by other users of the server, by implementing appropriate access controls on the server
- Generate a new session identifier and deactivate the old one periodically. (This can mitigate certain session hijacking scenarios where the original identifier was compromised)
- Generate a new session identifier if the connection security changes from HTTP to HTTPS, as can occur during authentication. Within an application, it is recommended to consistently utilize HTTPS rather than switching between HTTP to HTTPS.
- Supplement standard session management for sensitive server-side operations, like account management, by utilizing per-session strong random tokens or parameters. This method can be used to prevent Cross Site Request Forgery attacks
- Supplement standard session management for highly sensitive or critical operations by utilizing per-request, as opposed to per-session, strong random tokens or parameters
- Set the "secure" attribute for cookies transmitted over an TLS connection
- Set cookies with the HTTP Only attribute, unless you specifically require client-side scripts within your application to read or set a cookie's value

3.5 **Access Control:**

- Use only trusted system objects, e.g., server-side session objects, for making access authorization decisions
- Use a single site-wide component to check access authorization. This includes libraries that call external authorization services
- Access controls should fail securely
- Deny all access if the application cannot access its security configuration information
- Enforce authorization controls on every request, including those made by server-side scripts, "includes" and requests from rich client-side technologies like AJAX and Flash
- Segregate privileged logic from other application code
- Restrict access to files or other resources, including those outside the application's direct control, to only authorized users
- Restrict access to protected URLs to only authorized users

- Restrict access to protected functions to only authorized users
- Restrict direct object references to only authorized users
- Restrict access to services to only authorized users
- Restrict access to application data to only authorized users
- Restrict access to user and data attributes and policy information used by access controls
- Restrict access security-relevant configuration information to only authorized users
- Server-side implementation and presentation layer representations of access control rules must match
- If *state data* must be stored on the client, use encryption and integrity checking on the server side to catch state tampering.
- Enforce application logic flows to comply with business rules
- Limit the number of transactions a single user or device can perform in a given period. The transactions/time should be above the actual business requirement, but low enough to deter automated attacks
- Use the "referrer" header as a supplemental check only, it should never be the sole authorization check, as it is can be spoofed
- If long authenticated sessions are allowed, periodically re-validate a user's authorization to ensure that their privileges have not changed and if they have, log the user out and force them to re-authenticate
- Implement account auditing and enforce the disabling of unused accounts (e.g., After no more than 30 days from the expiration of an account's password.)
- The application must support disabling of accounts and terminating sessions when authorization ceases (e.g., Changes to role, employment status, business process, etc.)
- Service accounts or accounts supporting connections to or from external systems should have the least privilege possible
- Create an Access Control Policy to document an application's business rules, data types and access authorization criteria and/or processes so that access can be properly provisioned and controlled. This includes identifying access requirements for both the data and system resources

3.6 **Miscellaneous:**

- Application should display its last login date & time
- Idle session timeout should be configured 15 mins
- Account lockout should be configured for maximum five wrong attempts.
- Captcha should be implemented in all internets facing application or any relevant solution to protect brute forcing attacks.
- Force password change functionality should be present for first time login into the application.

- Auto logout should be configured after change password functionality.

3.7 Error Handling and Logging:

- Do not disclose sensitive information in error responses, including system details, session identifiers or account information
- Use error handlers that do not display debugging or stack trace information
- Implement generic error messages and use custom error pages
- The application should handle application errors and not rely on the server configuration
- Properly free allocated memory when error conditions occur
- Error handling logic associated with security controls should deny access by default
- All logging controls should be implemented on a trusted system (e.g., The server)
- Logging controls should support both success and failure of specified security events
- Ensure logs contain important *log event data*
- Ensure log entries that include un-trusted data will not execute as code in the intended log viewing interface or software
- Restrict access to logs to only authorized individuals
- Utilize a master routine for all logging operations
- Do not store sensitive information in logs, including unnecessary system details, session identifiers or passwords
- Ensure that a mechanism exists to conduct log analysis
- Log all input validation failures
- Log all authentication attempts, especially failures
- Log all access control failures
- Log all apparent tampering events, including unexpected changes to state data
- Log attempts to connect with invalid or expired session tokens
- Log all system exceptions
- Log all administrative functions, including changes to the security configuration settings
- Log all backend TLS connection failures
- Log cryptographic module failures
- Use a cryptographic hash function to validate log entry integrity

3.8 **Data Protection:**

- Implement least privilege, restrict users to only the functionality, data and system information that is required to perform their tasks
- Protect all cached or temporary copies of sensitive data stored on the server from unauthorized access and purge those temporary working files a soon as they are no longer required.
- Encrypt highly sensitive stored information, like authentication verification data, even on the server side. Always use well vetted algorithms, see "Cryptographic Practices" for additional guidance
- Protect server-side source-code from being downloaded by a user
- Do not store passwords, connection strings or other sensitive information in clear text or in any non-cryptographically secure manner on the client side. This includes embedding in insecure formats like: MS view state, Adobe flash or compiled code
- Remove comments in user accessible production code that may reveal backend system or other sensitive information
- Remove unnecessary application and system documentation as this can reveal useful information to attackers
- Do not include sensitive information in HTTP GET request parameters
- Disable auto complete features on forms expected to contain sensitive information, including authentication
- Disable client-side caching on pages containing sensitive information. Cache-Control: no-store, may be used in conjunction with the HTTP header control "Pragma: no-cache", which is less effective, but is HTTP/1.0 backward compatible
- The application should support the removal of sensitive data when that data is no longer required. (e.g. personal information or certain financial data)
- Implement appropriate access controls for sensitive data stored on the server. This includes cached data, temporary files and data that should be accessible only by specific system users

3.9 **Communication Security:**

- Implement encryption for the transmission of all sensitive information over internet. This should include TLS for protecting the connection and may be supplemented by discrete encryption of sensitive files or non-HTTP based connections
- TLS certificates should be valid and have the correct domain name, not be expired, and be installed with intermediate certificates when required
- Failed TLS connections should not fall back to an insecure connection
- Utilize TLS connections for all content requiring authenticated access and for all other sensitive information

- Utilize TLS for connections to external systems that involve sensitive information or functions
- Utilize a single standard TLS implementation that is configured appropriately
- Specify character encodings for all connections
- Filter parameters containing sensitive information from the HTTP referrer, when linking to external sites

3.10 **System Configuration:**

- Ensure servers, frameworks and system components are running the latest approved version
- Ensure servers, frameworks and system components have all patches issued for the version in use
- Ensure the server OS are with latest distribution with support available to receive all security patches.
- There should not be having any dependencies in future Upgradation or security updates while choosing any framework and system components. Feasibility should be checked at initial stage. In case
- Turn off directory listings
- Restrict the web server, process and service accounts to the least privileges possible
- When exceptions occur, fail securely and have custom error page.
- Remove all unnecessary functionality and files
- Remove all middleware version from HTTP headers.
- Remove test code or any functionality not intended for production, prior to deployment
- Prevent disclosure of your directory structure in the robots.txt file by placing directories not intended for public indexing into an isolated parent directory. Then "Disallow" that entire parent directory in the robots.txt file rather than disallowing each individual directory
- Define which HTTP methods, Get or Post, the application will support and whether it will be handled differently in different pages in the application
- Additional service components like SSH, OpenSSL, OpenSSH SFTP, SMB etc should be updated with latest release and future upgradable.
- Disable unnecessary HTTP methods, such as WebDAV extensions. If an extended HTTP method that supports file handling is required, utilize a well-vetted authentication mechanism
- If the web server handles both HTTP 1.0 and 1.1, ensure that both are configured in a similar manor or ensure that you understand any difference that may exist (e.g., handling of extended HTTP methods)
- Restrict the http method to GET and POST, specify in case other methods are required.

- Remove unnecessary information from HTTP response headers related to the OS, web-server version and application frameworks
- Remove all unnecessary default pages and directories. Restrict all default admin login pages.
- Always use upgraded middleware packages.
- The security configuration store for the application should be able to be output in human readable form to support auditing
- Implement an asset management system and register system components and software in it
- Isolate development environments from the production network and provide access only to authorized development and test groups. Development environments are often configured less securely than production environments and attackers may use this difference to discover shared weaknesses or as an avenue for exploitation
- Implement a software change control system to manage and record changes to the code both in development and production
- Implement OWASP security HTTP headers during configuration.
- Implement cookie attributes during configuration.

3.11 Database Security:

- Use strongly typed *parameterized queries*
- Utilize input validation and output encoding and be sure to address meta characters. If these fail, do not run the database command
- Ensure that variables are strongly typed
- The application should use the lowest possible level of privilege when accessing the database
- Use secure credentials for database access
- Connection strings should not be hard coded within the application. Connection strings should be stored in a separate configuration file on a trusted system, and they should be encrypted.
- Use stored procedures to abstract data access and allow for the removal of permissions to the base tables in the database
- Close the connection as soon as possible
- Remove or change all default database administrative passwords. Utilize strong passwords/phrases or implement multi-factor authentication
- Turn off all unnecessary database functionality (e.g., unnecessary stored procedures or services, utility packages, install only the minimum set of features and options required (surface area reduction))
- Remove unnecessary default vendor content (e.g., sample schemas)
- Disable any default accounts that are not required to support business requirements

- The application should connect to the database with different credentials for every trust distinction (e.g., user, read-only user, guest, administrators)

3.12 File Management:

- Do not pass user supplied data directly to any dynamic include function
- Require authentication before allowing a file to be uploaded
- Limit the type of files that can be uploaded to only those types that are needed for business purposes
- Validate uploaded files are the expected type by checking file headers. Checking for file type by extension alone is not sufficient
- Do not save files in the same web context as the application. Files should either go to the content server or in the database.
- Prevent or restrict the uploading of any file that may be interpreted by the web server.
- Turn off execution privileges on file upload directories
- Implement safe uploading in UNIX by mounting the targeted file directory as a logical drive using the associated path or the chrooted environment
- When referencing existing files, use a whitelist of allowed file names and types. Validate the value of the parameter being passed and if it does not match one of the expected values, either reject it or use a hard coded default file value for the content instead
- Do not pass user supplied data into a dynamic redirect. If this must be allowed, then the redirect should accept only validated, relative path URLs
- Do not pass directory or file paths, use index values mapped to pre-defined list of paths
- Never send the absolute file path to the client
- Ensure application files and resources are read-only
- Scan user uploaded files for viruses and malware

3.13 Memory Management:

- Utilize input and output control for un-trusted data
- Double check that the buffer is as large as specified
- When using functions that accept a number of bytes to copy, such as strncpy(), be aware that if the destination buffer size is equal to the source buffer size, it may not NULL-terminate the string
- Check buffer boundaries if calling the function in a loop and make sure there is no danger of writing past the allocated space

- Truncate all input strings to a reasonable length before passing them to the copy and concatenation functions
- Specifically close resources don't rely on garbage collection. (e.g., connection objects, file handles, etc.)
- Use non-executable stacks when available
- Avoid the use of known vulnerable functions (e.g., printf, strcat, strcpy etc.)
- Properly free allocated memory upon the completion of functions and at all exit points

4. API Security Best Practices

4.1 **Authentication**

- Don't use Basic Auth Use standard authentication (e.g., JWT, OAuth).
- Don't reinvent the wheel in Authentication, token generation, password storage. Use the standards.
- Use Max Retry and jail features in Login.
- Use encryption on all sensitive data.

4.2 **JWT (JSON Web Token)**

- Use a random complicated key (JWT Secret) to make brute forcing the token very hard.
- Don't extract the algorithm from the payload. Force the algorithm in the backend (HS256 or RS256).
- Make token expiration (TTL, RTTL) as short as possible.
- Don't store sensitive data in the JWT payload, it can be decoded easily.

4.3 **OAuth**

- Always validate redirect Uri server-side to allow only whitelisted URLs.
- Always try to exchange for code and not tokens (don't allow response type=token).
- Use state parameter with a random hash to prevent CSRF on the OAuth authentication process.
- Define the default scope and validate scope parameters for each application.

4.4 Access

- Limit requests (Throttling) to avoid DDoS / brute-force attacks.
- Use HTTPS on server side to avoid MITM (Man in the Middle Attack).
- Use HSTS header with SSL to avoid SSL Strip attack.

4.5 **Input**

- Use the proper HTTP method according to the operation: GET (read), POST (create), PUT/PATCH (replace/update), and DELETE (to delete a record), and respond with 405 Method Not Allowed if the requested method isn't appropriate for the requested resource.

- Validate content-type on request Accept header (Content Negotiation) to allow only your supported format (e.g. application/xml, application/json, etc) and respond with 406 Not Acceptable response if not matched.
- Validate content-type of posted data as you accept (e.g. application/x-www-form-urlencoded, multipart/form-data, application/json, etc).
- Validate User input to avoid common vulnerabilities (e.g. XSS, SQL-Injection, Remote Code Execution, etc).
- Don't use any sensitive data (credentials, Passwords, security tokens, or API keys) in the URL, but use standard Authorization header.
- Use an API Gateway service to enable caching, Rate Limit policies (e.g. Quota, Spike Arrest, Concurrent Rate Limit) and deploy APIs resources dynamically.

4.6 **Processing**

- Check if all the endpoints are protected behind authentication to avoid broken authentication process.
- User own resource ID should be avoided. Use /me/orders instead of /user/654321/orders.
- Don't auto-increment IDs. Use UUID instead.
- If you are parsing XML files, make sure entity parsing is not enabled to avoid XXE (XML external entity attack).
- If you are parsing XML files, make sure entity expansion is not enabled to avoid Billion Laughs/XML bomb via exponential entity expansion attack.
- Use a CDN for file uploads.
- If you are dealing with huge amount of data, use Workers and Queues to process as much as possible in background and return response fast to avoid HTTP Blocking.
- Do not forget to turn the DEBUG mode OFF.

4.7 **Output**

- Send X-Content-Type-Options: nosniff header.
- Send X-Frame-Options: deny header.
- Send Content-Security-Policy: default-src 'none' header.
- Remove fingerprinting headers - X-Powered-By, Server, X-AspNet-Version etc.
- Force content-type for your response, if you return application/json then your response content-type is application/json.
- Don't return sensitive data like credentials, Passwords, security tokens.
- Return the proper status code according to the operation completed. (e.g. 200 OK, 400 Bad Request, 401 Unauthorized, 405 Method Not Allowed, etc).

4.8 CI & CD

- Audit your design and implementation with unit/integration tests coverage.
- Use a code review process and disregard self-approval.
- Ensure that all components of your services are statically scanned by AV software before push to production, including vendor libraries and other dependencies.
- Design a rollback solution for deployments.

5. Web Service Best Practices (SOAP & REST)

5.1 Input Validation

- Input to Web methods is constrained and validated for type, length, format, and range.
- Input data sanitization is only performed in addition to constraining input data.
- XML input data is validated based on an agreed schema.

5.2 Authentication

- Web services that support restricted operations or provide sensitive data support authentication.
- If plain text credentials are passed in SOAP headers, SOAP messages are only passed over encrypted communication channels, for example, using SSL.
- Basic authentication is only used over an encrypted communication channel.
- Authentication mechanisms that use SOAP headers are based on Web Services Security (WS Security) using the Web Services Enhancements WSE).

5.3 Authorization

- Web services that support restricted operations or provide sensitive data support authorization.
- Where appropriate, access to Web service is restricted using URL authorization or file authorization if Windows authentication is used.
- Where appropriate, access to publicly accessible Web methods is restricted using declarative principal permission demands.

5.4 Sensitive Data

- Sensitive data in Web service SOAP messages is encrypted using XML encryption OR messages are only passed over encrypted communication channels (for example, using SSL.)

5.5 Parameter Manipulation

- If parameter manipulation is a concern (particularly where messages are routed through multiple intermediary nodes across multiple network links). Messages are digitally signed to ensure that they cannot be tampered with.

5.6 Exception Management

- Structured exception handling is used when implementing Web services.
- Exception details are logged (except for private data, such as passwords).
- Soap Exceptions are thrown and returned to the client using the standard <Fault> SOAP element.
- If application-level exception handling is required a custom SOAP extension is used.

5.7 Auditing and Logging

- The Web service logs transactions and key operations.

5.8 Proxy Considerations

- The endpoint address in Web Services Description Language (WSDL) is checked for validity.
- The URL Behaviour property of the Web reference is set to dynamic for added flexibility.

5.9 Administration Considerations

- Unnecessary Web service protocols, including HTTP GET and HTTP POST, are disabled.
- The documentation protocol is disabled if you do not want to support the dynamic generation of WSDL.
- The Web service runs using a least-privileged process account (configured through the <processModel> element in Machine.config.)
- Custom accounts are encrypted by using Aspnet_setref.exe.
- Tracing is disabled with:
  <trace enabled="false" />
- Debug compilations are disabled with:
  <compilation debug="false" explicit="true" defaultLanguage="vb">

6. Hardening Review:

- Hardening Points review is performed on regular basis as per VAPT Policy.

7. Exceptions:

- CISO sign off will be required in case of exception required to above applicable controls.

8. Policy Compliance

8.1Compliance Measurement

The Infosec Team will verify compliance to this policy through various methods, including but not limited to, business tool reports, internal and external audits, and feedback to the policy owner.

8.2Exceptions

Any exception to the policy must be approved by the Infosec team in advance.

8.3 Non-Compliance

An employee found to have violated this policy may be subject to disciplinary action, up to and including termination of employment.

9. Related References:

ISO Standard Reference	ISO 27001:2013 A.9.4.5 Access control to program source code
IRDAI Cyber Security Reference	IRDAI Cyber Security Guidelines 5.14.1 Establish security and access control policies & procedure
Internal Document Reference	• IRDAI Guidelines • Source Code Review Process • Secure SDLC Policy

10. Measurement Metrices

Sr. No.	Particulars/ Reports	Frequency
1.	Review of Application API and Secure Code Guidelines for updates	Annually
2.	Compliance to API and Secure Code Guidelines	Ongoing

11. Contact:

Chief Information Security Officer
Tata AIG General Insurance Company Limited
Peninsula Business Park, Tower A,
15th Floor, G.K.Marg,
Lower Parel,
Mumbai - 400 013,
Maharashtra, INDIA.
CIN Number: U85110MH2000PLC128425
infosec@tataaig.com

12. Non-disclosure:

Internal Document – Circulation Restricted

The information (data) contained in all sheets of this ISMS Policy constitutes information that is trade specific and confidential or privileged. It is furnished to TAGIC employees in confidence with the understanding that it will not, without permission of the offer or, be used or disclosed for other than evaluation purposes.

If you are not the intended recipient of this document as mentioned in the distribution list, kindly delete this document, and inform the approver at infosec@tataaig.com